



따릉이 대여량 예측 모델 개발

장인서 백재은 유연 정윤지

목차

#1 프로젝트 소개

#2 EDA

#3 전처리

#4 모델링

#5 최종 결과

01 프로젝트 개요



프로젝트 개요

서울시 지역구별 따릉이 대여량 예측 모델 개발
2022년의 서울시 지역구별 일별 따릉이 대여량에 대한 회귀 예측



2022 UOS 빅데이터 알고리즘 경진대회

출처 : DAICON - Data Science Competition

 dacon.io

프로젝트 개요 - 제공 데이터

train.csvsample_submission.csv

Views

Grid view

Hide fields

Filter

Group

Sort

일시

광진구

동대문구

성동구

중랑구

1

20180101

0.592

0.368

0.580

0.162

2

20180102

0.840

0.614

1.034

0.260

3

20180103

0.828

0.576

0.952

0.288

4

20180104

0.792

0.542

0.914

0.292

5

20180105

0.818

0.602

0.994

0.308

6

20180106

0.618

0.522

0.696

0.228

7

20180107

0.598

0.478

0.606

0.238

8

20180108

0.742

0.528

0.848

0.202

9

20180109

0.652

0.488

0.838

0.206

10

20180110

0.500

0.406

0.636

0.160

10 records

Sum 201801055

Sum 6.980

Sum 5.124

Sum 8.098

Sum 2.344

Airtable

Copy base

View larger version

02 EDA



EDA - data cleaning & data formatting

한글 → 영어 변환

```
1 train_df = train_df.rename(columns={
2     '광진구': 'Gwangjin',
3     '동대문구': 'Dongdaemun',
4     '성동구': 'Seongdong',
5     '종랑구': 'Jungnang'
6 })
```

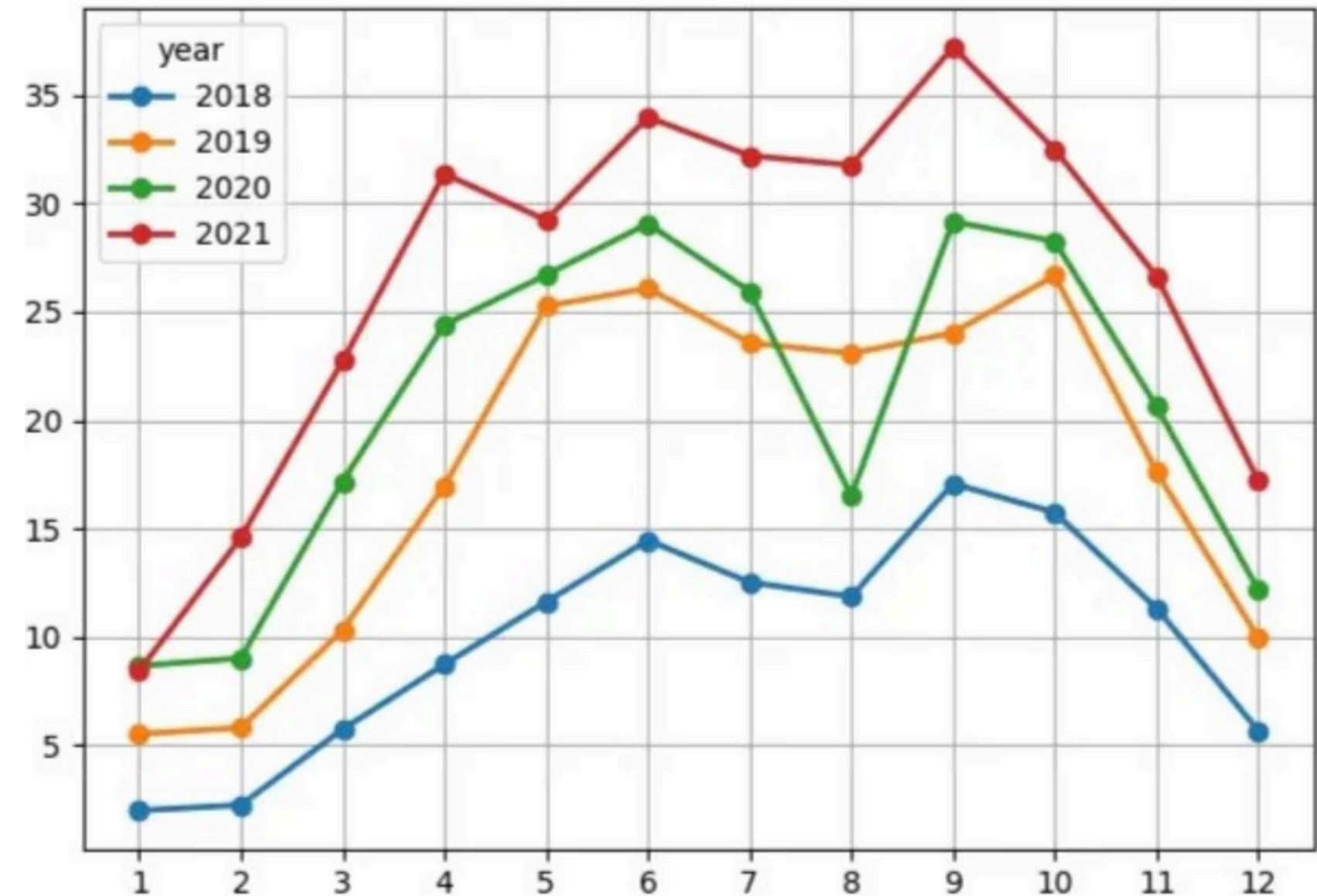
datetime 형식 변환

```
df['date'] = pd.to_datetime(df['date'], format='%Y%m%d')
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['weekday'] = df['date'].dt.weekday
df['is_weekend'] = df['weekday'].isin([5,6])
```

trend 분석 - 월별 평균 대여량

월별 경향성

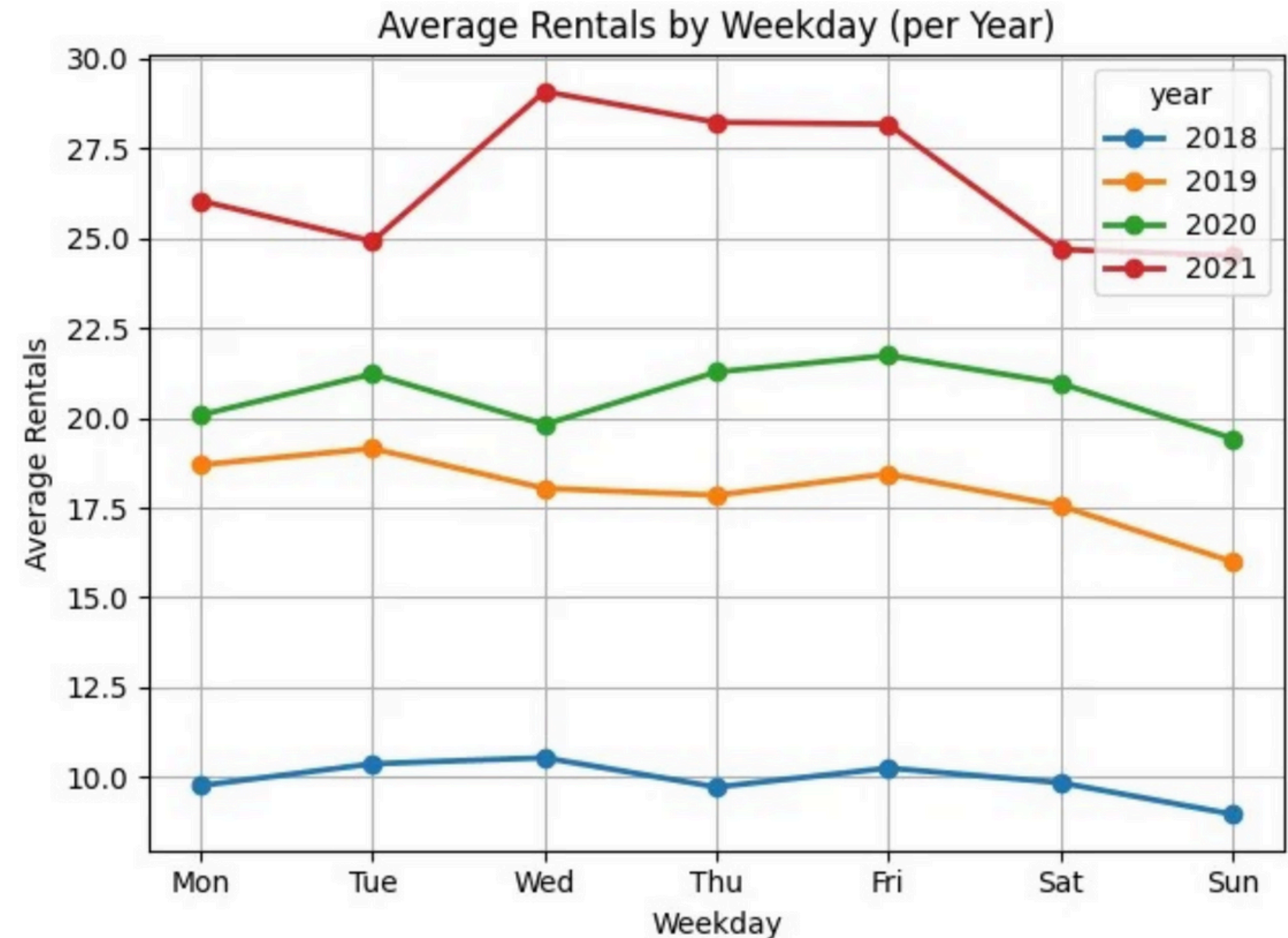
- 3월부터 대여량이 증가
- 9월이 정점이며, 이후 겨울철(12~2월)의 대여량은 감소
→ 뚜렷한 계절성 패턴을 확인
- 8월: 장마, 폭염 등의 영향으로 일시적 감소
- 9월: 연중 최고치 기록(2021년 9월, 최대 대여량 기록)
- 2021년: 전반적으로 모든 월에서 대여량이 높음
→ 코로나 이후 생활 패턴 변화 및 따릉이 인프라 개선 영향



trend 분석 - 요일별 평균 대여량

요일별 경향성

- 평일/주말: 평일 대여량이 높음(일요일 대여량 최저)
- 수~금: 예측 모델링에서 가장 중요한 축
- 요일에 따라 수요 편차가 존재
→ 모델 입력 변수에 요일 정보를 포함



공휴일 데이터

import holidays

```
import holidays

# 1. 한국 공휴일 객체 생성
kr_holidays = holidays.KR(years=range(2018, 2023)) # 2018~2022년

# 2. df 복사
df_holiday = df.copy()

# 3. 'date' 컬럼이 datetime 타입인지 확인 및 변환
df_holiday['date'] = pd.to_datetime(df_holiday['date'])
```

	date	Gwangjin	Dongdaemun	Seongdong	Jungnang	year	month	day	weekday	is_weekend	holiday	is_holiday
0	2018-01-01	0.592	0.368	0.580	0.162	2018	1	1	0	False	New Year's Day	True
1	2018-01-02	0.840	0.614	1.034	0.260	2018	1	2	1	False	not_holiday	False
2	2018-01-03	0.828	0.576	0.952	0.288	2018	1	3	2	False	not_holiday	False
3	2018-01-04	0.792	0.542	0.914	0.292	2018	1	4	3	False	not_holiday	False
4	2018-01-05	0.818	0.602	0.994	0.308	2018	1	5	4	False	not_holiday	False

공휴일 데이터 - 공휴일 유형

어린이날 (Children's Day)

- 광진구: 2020~21년에 뚜렷한 증가세, 이전에는 혼잡 양상.
- 동대문구: 전반적으로 큰 변화 없으며 2021년에만 다소 증가.
- 성동구: 매년 증가하며 특히 2021년에 급증.
- 중랑구: 전체적으로 미세한 증가로 안정적 수요 유지.

크리스마스 (Christmas)

- 광진구: 해마다 점점 감소하며 2021년에 급감.
- 동대문구: 4년 연속 감소, 2021년에 큰 하락.
- 성동구: 꾸준히 감소, 2021년에는 최대 감소폭 기록.
- 중랑구: 소폭이지만 일관된 감소 추세.

추석 (Chuseok)

- 광진구: 매년 감소 경향 뚜렷.
- 동대문구: 큰 폭의 하락 지속.
- 성동구: 꾸준한 감소세 유지.
- 중랑구: 소폭이지만 지속적으로 하락.

설날 (Seollal)

- 전 지역구: 전체적으로 큰 변화 없이 소폭 감소 또는 유지.

광복절 (Liberation Day)

- 전 지역구: 대부분 유지 혹은 미미한 감소로 뚜렷한 변화 없음.

공휴일 효과 분석

- 공휴일 효과는 공휴일의 성격에 따라 상이
→ 명절/크리스마스: 대여량 수요의 뚜렷한 감소
→ 어린이날 등 활동 중심 공휴일: 대여 수요 증가
- 단순 공휴일 여부만으로는 예측 한계가 존재

공휴일 데이터 - 급증/급감

탐지 방식

- 각 날짜의 대여량에서 3일 중심 이동평균을 뺀 잔차를 계산
- 잔차가 큰 상위 10일을 급증일(수요 이상치)로 정의

주요 관찰 결과

- 2개 이상의 지역에서 동시 급증한 날짜 존재
- 날씨, 이벤트 등 외부 요인의 영향 가능성 존재

모델링 활용

- 급증일 여부를 나타내는 binary feature를 생성

광진구 잔차가 가장 큰 날 (급증일 후보) 및 공휴일 여부:

	date	Gwangjin	gwangjin_residual	holiday	is_holiday
1329	2021-08-22	11.918	5.234000	not_holiday	False
838	2020-04-18	11.258	5.116000	not_holiday	False
1377	2021-10-09	11.336	4.372667	Hangul Day	True
618	2019-09-11	9.790	4.068000	not_holiday	False
981	2020-09-08	9.450	3.950667	not_holiday	False
1337	2021-08-30	13.242	3.863333	not_holiday	False
1220	2021-05-05	12.446	3.618000	Children's Day	True
867	2020-05-17	10.754	3.494000	not_holiday	False
1248	2021-06-02	11.884	3.274000	not_holiday	False
986	2020-09-13	11.150	3.198000	not_holiday	False

지역별 인구수 데이터

지역별 인구 증감 수준 - 추가 데이터(공공 데이터 포털)

대여량과 인구 간 교차 상관관계

병합된 데이터:

	Gwangjin_rent	Dongdaemun_rent	Seongdong_rent	Jungnang_rent
year				
2018	3.497041	1.969063	3.058011	1.399266
2019	6.619107	3.676953	4.986762	2.679167
2020	6.553978	4.626852	5.767557	3.695661
2021	8.165704	6.660734	6.917874	4.779370

	Dongdaemun_pop	Gwangjin_pop	Jungnang_pop	Seongdong_pop
year				
2018	348052	355559	403209	308221
2019	346194	351350	397015	300889
2020	342837	346682	394702	293556
2021	337400	339996	387350	285990

	Dongdaemun	Gwangjin	Jungnang	Seongdong
Gwangjin	-0.853408	-0.900477	-0.949932	-0.919813
Dongdaemun	-0.979356	-0.994077	-0.998491	-0.992111
Seongdong	-0.929903	-0.965684	-0.980536	-0.982363
Jungnang	-0.965759	-0.989263	-0.985117	-0.998575

→ 모든 지역에서 음의 상관관계

→ 인구수에 따라 따릉이 대여소 및 따릉이를 배치하는 것은 적절하지 않을 수 있음

03 전처리



모델 선정

초기 목적: 회귀 모델 + 시계열 모델 → 앙상블

- Catboost
- LightGBM
- Prophet
- Arima, Sarima

시계열 모델 사용 어려움 판단 ⇒ **시계열 모델**을 회귀 모델로 수정

- Catboost
- LightGBM
- **KNN**
- **RandomForest**

** 모델별 성능에 따라 전처리, feature 차이 존재

공통 전처리

pd.to_datetime

날짜 → 연, 월, 요일, 주말 여부

```
[ ] df['date'] = pd.to_datetime(df['date'], format='%Y%m%d')
    df['year'] = df['date'].dt.year
    df['month'] = df['date'].dt.month
    df['day'] = df['date'].dt.day
    df['weekday'] = df['date'].dt.weekday
    df['is_weekend'] = df['weekday'].isin([5,6])
```

import holidays

공휴일 데이터 생성 & 임시 공휴일 및 선거일 직접 추가

```
▶ import holidays

kr_holidays = holidays.KR(years=range(2018, 2023))

# 임시공휴일 및 선거일 직접 추가
extra_holidays = {
    '2018-06-13': '제7회 전국동시지방선거',
    '2020-04-15': '제21대 국회의원선거',
    '2020-08-17': '임시공휴일' }

for date, name in extra_holidays.items():
    kr_holidays[date] = name

df['holiday'] = df['date'].apply(lambda x: kr_holidays.get(x) if x in kr_holidays else 'not_holiday')
df['is_holiday'] = df['holiday'] != 'not_holiday'
```

모델별 전처리

CatBoost) 활동형/휴식형 공휴일 구분

```
# 활동형/휴식형 공휴일 구분
activity_holidays = ['어린이날']
rest_holidays = ['설날', '추석', '크리스마스']
train_df['is_activity_holiday'] = train_df['holiday_name'].isin(activity_holidays).astype(int)
train_df['is_rest_holiday'] = train_df['holiday_name'].isin(rest_holidays).astype(int)
```

KNN, LightGBM, RandomForest) 계절 feature 생성

```
def get_season(month):
    if month in [3, 4, 5]: return 'spring'
    elif month in [6, 7, 8]: return 'summer'
    elif month in [9, 10, 11]: return 'fall'
    else: return 'winter'
```

계절 feature 원핫 인코딩

```
df = pd.get_dummies(df, columns=['season'], drop_first=True)
```


04 모델링



CatBoost

1. Optuna를 이용하여
best_params 찾기
2. 구별 best_params 기반 →
Optuna 탐색 범위 재설정,
fine tuning 반복
3. 구별 최소 RMSE 파라미터로
학습 진행

```
# 모델 학습 및 검증
```

```
# 예시: target_cols와 해당 구별로 train/val 데이터, cat_features는 미리 준비  
# target_cols = ['Gwangjin', 'Dongdaemun', 'Seongdong', 'Jungnang']
```

```
# '최종 best params' 사전
```

```
best_params_dict = {
```

```
'Gwangjin': {'learning_rate': 0.07463933708677424, 'depth': 5, 'l2_leaf_reg': 1.074777222256696,  
'subsample': 0.805469615390896, 'colsample_bylevel': 0.7879853811582616},  
'Dongdaemun': {'learning_rate': 0.05090532965560029, 'depth': 4, 'l2_leaf_reg': 2.4336961951246585,  
'subsample': 0.9860732586553317, 'colsample_bylevel': 0.8329465092538842},  
'Seongdong': {'learning_rate': 0.06865667868137201, 'depth': 5, 'l2_leaf_reg': 3.548575891319757,  
'subsample': 0.7118705479838677, 'colsample_bylevel': 0.7334892361378472},  
'Jungnang': {'learning_rate': 0.05691151851732044, 'depth': 4, 'l2_leaf_reg': 2.5595777632181074,  
'subsample': 0.9751001756331396, 'colsample_bylevel': 0.5490555159536538}
```

```
}
```

```
results_summary = []
```

```
all_valid_preds = {}
```

CatBoost

4. 모델 학습 및 검증

- 구별 독립적인 CatBoost 모델 학습 진행
- 학습 시 검증 데이터를 활용,
early stopping (50회 미개선 시 종료)
- 학습 이후, 검증 데이터에 대한 예측 수행
→ 예측값 저장

5. 성능 평가 및 결과 요약

- 검증 데이터에 대한 RMSE 계산
- 구별 RMSE 및 최적 하이퍼파라미터를
딕셔너리 및 DataFrame으로 정리

```
for target in target_cols:
    params = {
        "iterations": 1000,
        "learning_rate": best_params_dict[target]["learning_rate"],
        "depth": best_params_dict[target]["depth"],
        "l2_leaf_reg": best_params_dict[target]["l2_leaf_reg"],
        "subsample": best_params_dict[target]["subsample"],
        "colsample_bylevel": best_params_dict[target]["colsample_bylevel"],
        "random_seed": 42,
        "cat_features": cat_features,
        "loss_function": "RMSE",
        "verbose": 0
    }

    y_train = train_df.loc[train_idx, target]
    y_val = train_df.loc[val_idx, target]
    model = cb.CatBoostRegressor(**params)
    model.fit(X_train, y_train, eval_set=(X_val, y_val), early_stopping_rounds=50)
    preds = model.predict(X_val)

    rmse = np.sqrt(mean_squared_error(y_val, preds))
    results_summary.append({
        '구': target,
        'Best RMSE': round(rmse, 4),
        'Best Params': best_params_dict[target]
    })

    all_valid_preds[target] = preds

# 구별 성능 요약
summary_df = pd.DataFrame(results_summary)
print("\n=== 모든 구별 CatBoost Best RMSE & Params 요약 ===")
print(summary_df.to_markdown(index=False))
```

Light GBM

1. LightGBM 회귀 모델을
시계열 데이터에 맞춰 학습·평가
2. 날짜·타깃 열 제외 후
학습/테스트 세트 구성
3. GridSearchCV로 RMSE 기준
최적 파라미터 탐색
4. TimeSeriesSplit으로
시계열 교차 검증 수행
5. MAE, RMSE, 예측값,
최적 파라미터 반환

```
# grid search
def train_eval_one_region(df_all: pd.DataFrame, target_col: str, feature_cols: list[str],
                           train_end_year: int, test_year: int):
    use_cols = ['date', target_col] + feature_cols
    dfg = df_all[use_cols].copy()
    dfg = dfg.rename(columns={target_col: 'rental'})

    train_df = dfg[dfg['date'].dt.year <= train_end_year]
    test_df = dfg[dfg['date'].dt.year == test_year]

    X_train = train_df.drop(columns=['date', 'rental'])
    y_train = train_df['rental']
    X_test = test_df.drop(columns=['date', 'rental'])
    y_test = test_df['rental'] if 'rental' in test_df.columns else None

    base = lgb.LGBMRegressor(random_state=42)
    tscv = TimeSeriesSplit(n_splits=5)
    grid = GridSearchCV(base, PARAM_GRID, scoring=rmse_scorer, cv=tscv, n_jobs=-1, verbose=1)
    grid.fit(X_train, y_train)

    model = grid.best_estimator_
    y_pred = model.predict(X_test) if len(X_test) else np.array([])
    mae = mean_absolute_error(y_test, y_pred) if y_test is not None and len(y_pred)==len(y_test) else None
    rmse = rmse_score(y_test, y_pred) if y_test is not None and len(y_pred)==len(y_test) else None

    return {
        'model': model,
        'best_params': grid.best_params_,
        'cv_rmse': -grid.best_score_,
        'test_mae': mae,
        'test_rmse': rmse,
        'y_pred': y_pred
    }
```


Light GBM

1. Optuna를 이용해
LightGBM 하이퍼 파라미터 자동 최적화
2. 목표 함수에서 학습/검증 데이터와
파라미터 샘플링 정의
3. 모델 학습 후 검증 세트 RMSE 계산
4. 여러 trial 반복 실행
5. RMSE 최소 파라미터 조합 도출

```
# Optuna
def optuna_objective(trial, X_train, y_train, X_valid, y_valid):
    params = {
        'objective': 'regression', 'metric': 'rmse', 'random_state': 42,
        'n_estimators': trial.suggest_int('n_estimators', 300, 1500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'num_leaves': trial.suggest_int('num_leaves', 16, 256, log=True),
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 100),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0.0, 0.5),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.0, 0.5),
    }
    model = lgb.LGBMRegressor(**params)
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    return rmse_score(y_valid, pred)
```

Light GBM

```
# Grid vs Optuna
def train_eval_one_region_hybrid(df_all: pd.DataFrame, target_col: str, feature_cols: list[str],
                                  train_end_year: int, test_year: int,
                                  use_grid=True, use_optuna=True, n_trials=30):

    use_cols = ['date', target_col] + feature_cols
    dfg = df_all[use_cols].copy()
    dfg = dfg.rename(columns={target_col: 'rental'})

    train_df = dfg[dfg['date'].dt.year <= train_end_year]
    valid_df = dfg[dfg['date'].dt.year == test_year]

    X_train = train_df.drop(columns=['date', 'rental']); y_train = train_df['rental']
    X_valid = valid_df.drop(columns=['date', 'rental']); y_valid = valid_df['rental'] if 'rental' in valid_df.columns else None
```

1. Optuna를 이용해 LightGBM 하이퍼파라미터 자동 최적화
2. 목표 함수에서 학습·검증 데이터와 파라미터 샘플링 정의
3. 모델 학습 후 검증 세트 RMSE 계산
4. Optuna가 여러 trial 반복 실행
5. RMSE 최소인 파라미터 조합 도출

Light GBM

1. 특정 지역·기간 데이터
학습 및 평가
2. 날짜 조건으로 학습/검증
세트 분리
3. 지정된 특징(feature)만
학습에 사용
4. GridSearch 또는
Optuna 중 선택 가능
5. 모델 객체, 최적 파라미터,
검증 점수, 예측값 반환

```
# GridSearch
if use_grid:
    base = lgb.LGBMRegressor(random_state=42)
    tscv = TimeSeriesSplit(n_splits=5)
    grid = GridSearchCV(base, PARAM_GRID, scoring=rmse_scorer, cv=tscv, n_jobs=-1, verbose=1)
    grid.fit(X_train, y_train)
    model_g, params_g, cv_rmse_g = grid.best_estimator_, grid.best_params_, -grid.best_score_
else:
    model_g = lgb.LGBMRegressor(n_estimators=1200, learning_rate=0.03, subsample=0.9, colsample_bytree=0.9, random_state=42)
    model_g.fit(X_train, y_train); params_g, cv_rmse_g = None, None

has_valid = (y_valid is not None) and (len(X_valid) > 0) and (len(y_valid) > 0)
if not has_valid:
    return {'model': model_g, 'best_params': params_g, 'mode': 'grid' if use_grid else 'base',
            'cv_rmse': cv_rmse_g, 'test_mae': None, 'test_rmse': None, 'y_pred': np.array([])}

y_pred_g = model_g.predict(X_valid)
rmse_g = rmse_score(y_valid, y_pred_g); mae_g = mean_absolute_error(y_valid, y_pred_g)
best_model, best_desc, best_pred = model_g, ('grid' if use_grid else 'base', params_g, rmse_g, mae_g, cv_rmse_g), y_pred_g

# Optuna
if use_optuna and OPTUNA_AVAILABLE:
    study = optuna.create_study(direction='minimize')
    study.optimize(lambda t: optuna_objective(t, X_train, y_train, X_valid, y_valid),
                  n_trials=n_trials, show_progress_bar=False)
    if len(study.trials):
        params_o = study.best_params
        model_o = lgb.LGBMRegressor(random_state=42, **params_o)
        model_o.fit(X_train, y_train)
        y_pred_o = model_o.predict(X_valid)
        rmse_o = rmse_score(y_valid, y_pred_o); mae_o = mean_absolute_error(y_valid, y_pred_o)
        if rmse_o < best_desc[2]:
            best_model, best_desc, best_pred = model_o, ('optuna', params_o, rmse_o, mae_o, cv_rmse_g), y_pred_o

return {'model': best_model, 'best_params': best_desc[1], 'mode': best_desc[0],
        'cv_rmse': best_desc[4], 'test_mae': best_desc[3], 'test_rmse': best_desc[2], 'y_pred': best_pred}
```


KNN

- 파이프라인 정의: 스케일링 + KNN 회귀
- 하이퍼 파라미터 탐색 (grid search)
- 최적 모델로 예측 및 평가

```
# 파이프라인 정의: 스케일링 + KNN 회귀
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsRegressor()) ])

# 하이퍼 파라미터 탐색
param_grid = {'knn__n_neighbors': list(range(1, 51, 2)),
              'knn__weights': ['uniform', 'distance'],
              'knn__metric': ['euclidean', 'manhattan'],
              'knn__leaf_size': [10, 20, 30, 40, 50],
              'knn__p': [1, 2] }

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    verbose=1)

# 최적 모델로 예측 및 평가
grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
y_pred_G = best_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred_G)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_G))

print("최적 하이퍼파라미터:", grid_search.best_params_)
print(f"최고 CV RMSE: {-grid_search.best_score_:.4f}")
```

RandomForest

1. 최종 모델의 입력 변수 정의

- 모델의 입력으로 사용할 변수 목록 정의
- 계절&공휴일: 대여 수요 패턴에 강한 영향을 줄 수 있는 변수로 포함

```
# 최종 모델의 입력 변수 정의
features = ['year', 'month', 'day', 'weekday', 'is_weekend',
            'season_spring', 'season_summer', 'season_winter', 'is_holiday']
```

2. 모델 학습 및 평가

- 각 지역 구별 반복하여 모델 학습 및 평가

```
# 모델 학습 및 평가
results = []
for target in districts:
    X = df[features]
    y = df[target]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)

    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)

    results.append({'Region': target, 'MSE': mse, 'RMSE': rmse})

results_df = pd.DataFrame(results)
print("\n최종 모델 성능 (계절 + 공휴일 포함):")
print(results_df)
```

#모델별 성능(RMSE)

2018~2020년 데이터 → 2021년 대여량 예측

지역	CatBoost	KNN	RandomForest	LightGBM
광진구	2.9355	2.7697	1.7839	1.1753
동대문구	2.7365	1.5539	1.2071	0.8293
성동구	2.5493	2.0643	1.5098	1.0782
중랑구	1.7139	1.2685	0.9208	0.5352

⇒ 앙상블로 성능 향상

앙상블

가중평균

- gridsearch
- scipy optimize
- optuna optimize
- 모델별 성능 기반 가중치 설정

스태킹

- 모델 조합(9개) → 가능한 모든 조합
- 메타 모델 6개 (linear regression, random forest, catboost, xgboost, svr, elastic net)

앙상블 - 가중평균

grid search

지역	MAE	RMSE
광진구	1.8855	2.452
동대문구	1.7289	2.1977
성동구	1.7597	2.2325
중랑구	1.1891	1.4833

scipy optimize

지역	MAE	RMSE
광진구	1.7324	2.4327
동대문구	1.4299	1.872
성동구	1.5911	2.1985
중랑구	1.0447	1.3465

성능별 가중치

exp(-RMSE) 기반
RMSE: 2.3233
MAE : 1.9030

optuna optimize

지역	MAE	RMSE
광진구	1.7324	2.4326
동대문구	1.43	1.872
성동구	1.591	2.1984
중랑구	1.0447	1.3465

앙상블 - 스택킹

최적 성능 모델 조합 & 메타 모델

Best 성능!

모델 조합	메타 모델	광진구	동대문구	성동구	중랑구
lightGBM + randomforest + knn	XGBoost	0.6110 / 0.8376	0.4975 / 0.6778	0.4954 / 0.6907	0.3554 / 0.4754
all	XGBoost	0.6110 / 0.8376	0.4975 / 0.6778	0.4954 / 0.6907	0.3554 / 0.4754

지역별 최적 성능 조합

지역	모델 조합	메타 모델	MAE	RMSE
광진구	all	RandomForest	0.6832	0.9493
동대문구	all	CatBoost	0.5338	0.6817
성동구	all	CatBoost	0.6292	0.7904
중랑구	all	CatBoost	0.386	0.502

앙상블 - BEST

공통 학습 및 평가 프로세스

1. 각 모델 조합(combo_name)별로:

- 해당 메타 특성 데이터(X_meta)를 가져옴
- 4개 구별로:
 - 구별 컬럼 선택(col.endswith(f"_{gu}"))
 - 인덱스 정렬(reset_index(drop=True))
- 데이터 정합성 확보
 - 검증 타깃 데이터도 인덱스 맞춰 분리
 -

2. 메타 모델을 해당 구별 데이터로 학습

3. 학습된 모델로 자기 데이터에 대해 예측 후, MAE와 RMSE 계산

```
results_xgb = {}

for combo_name, X_meta in X_meta_dict.items():
    print(f"=== {combo_name} ===")
    results_xgb[combo_name] = {}
    xgb_meta = {}

    for gu in ['Gwangjin', 'Dongdaemun', 'Seongdong', 'Jungnang']:
        gu_cols = [col for col in X_meta.columns
                    if col.endswith(f"_{gu}")]
        X_meta_gu = X_meta[gu_cols].reset_index(drop=True)
        y_gu = y_valid[gu].reset_index(drop=True)

        meta_model = XGBRegressor(n_estimators=100, learning_rate=0.1,
                                   random_state=42, verbosity=0)
        meta_model.fit(X_meta_gu, y_gu)

        pred_gu = meta_model.predict(X_meta_gu)
        rmse = np.sqrt(mean_squared_error(y_gu, pred_gu))
        mae = mean_absolute_error(y_gu, pred_gu)

        xgb_meta[gu] = meta_model

    print(f"{gu} MAE: {mae:.4f}, RMSE: {rmse:.4f}")
    results_xgb[combo_name][gu] = {
        'MAE': mae,
        'RMSE': rmse}

print("")
```

05 최종 결과



최종 선정 앙상블 - XGBoost(all)

XGBoost(all) - 2022년 최종 예측

2021년 스택킹 메타 모델(XGBoost 등) 학습 및 저장

- 1) 각 구(광진구, 동대문구, 성동구, 중랑구) 별 XGBoost 등 메타 모델을 별도 학습
- 2) 학습 완료된 구별 메타모델을 저장

```
from joblib import dump

save_dir = '/content/drive/MyDrive/stacking_models/'

for gu, model in xgb_meta.items():
    dump(model, save_dir + f"xgb_meta_{gu}.joblib")
```

최종 선정 앙상블 - XGBoost(all)

2021년 저장된 메타 모델의 로드 및 2022년 예측

저장한 구별 메타 모델 파일을 load()로 호출

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

import os
print(os.listdir('/content/drive/MyDrive/stacking_models'))

from joblib import load

load_dir = '/content/drive/MyDrive/stacking_models/'

xgb_meta_loaded = {}
for gu in ['Gwangjin', 'Dongdaemun', 'Seongdong', 'Jungnang']:
    xgb_meta_loaded[gu] = load(load_dir + f"xgb_meta_{gu}.joblib")

print(xgb_meta_loaded.keys())
```


최종 선정 앙상블 - XGBoost(all)

```
### XGBoost

results_xgb = {}

for combo_name, X_meta in X_meta_dict.items():
    print(f"=== {combo_name} ===")
    results_xgb[combo_name] = {}

    for gu in ['Gwangjin', 'Dongdaemun', 'Seongdong', 'Jungnang']:
        gu_cols = [col for col in X_meta.columns if col.endswith(f"_{gu}")]
        X_meta_gu = X_meta[gu_cols].reset_index(drop=True)

        pred_gu = xgb_meta_loaded[gu].predict(X_meta_gu)

        results_xgb[combo_name][gu] = {
            'predictions': pred_gu
        }

        print(f"{gu} 예측값 샘플: {pred_gu[:5]}") # 예측값 일부 출력

print("")
```

2022년의 메타피처(X_meta_all_2022)에서
구별로 해당 칼럼만을 추출
불러온 모델로 predict 진행

예측 결과를 dict 형태로 result_xgb에 저장

스태킹 진행

최종 선정 앙상블 - XGBoost(all)

sample_submission.csv(제출파일 예시)의 날짜 범위에 맞추어 예측값 slicing

예측값을 바로 각 칼럼에 assign

index = False로 저장

```
# sample_submission.csv 로드 (334개 행 - 1월 1일~11월 30일)
submission = pd.read_csv('sample_submission.csv')

n = len(submission) # 334

# 예측값 365개 중 처음 334개만 사용
submission['광진구'] = results_xgb['all']['Gwangjin']['predictions'][:n]
submission['동대문구'] = results_xgb['all']['Dongdaemun']['predictions'][:n]
submission['성동구'] = results_xgb['all']['Seongdong']['predictions'][:n]
submission['중랑구'] = results_xgb['all']['Jungnang']['predictions'][:n]

submission.to_csv('submission.csv', index=False)
print("submission.csv 파일이 생성되었습니다 (11월 30일까지)")
```

submission.csv

일시	광진구	동대문구	성동구	종량구
20220101	3.7363133	2.7128289	2.3181784	2.7311277
20220102	3.9426272	2.4369464	2.5540264	2.4777446
20220103	3.3618972	2.9592023	2.7322361	2.3430114
20220104	2.2463562	2.5290236	2.2461927	2.7158337
20220105	2.2627232	3.2210143	2.688431	2.1854036
20220106	2.2463562	1.8552456	2.688431	1.810259
20220107	1.9451171	2.2791803	2.7287107	1.810259
20220108	3.126228	2.0113747	1.8409567	1.5155265
20220109	3.2719388	2.9592023	1.5131834	2.0544698
20220110	3.3158832	2.0113747	2.6094735	2.4777446
20220111	2.2463562	2.1373007	3.2111154	2.3118749
20220112	1.8669057	2.0113747	2.7873592	3.1229358
20220113	2.271618	2.034949	2.9180655	1.810259
20220114	2.2704296	2.2426414	3.0668833	1.3159062
20220115	3.0246937	2.9592023	1.8665038	3.0217018
20220116	3.9426272	2.4166632	1.8665038	2.5254836
20220117	1.9485415	1.9124731	2.7463527	1.3702234
20220118	1.9287496	1.9124731	3.0975223	1.810259
20220119	2.2704296	1.8940259	3.4003432	1.4152566
20220120	3.6948066	2.2426414	2.6562912	1.9426726
20220121	2.1835976	2.2426414	3.9940665	2.2619789
20220122	3.126228	3.4673877	3.491553	3.2623243
20220123	3.2719388	2.7128289	3.491553	2.7158337
20220124	2.3823442	1.9365151	2.8150833	1.7109085
20220125	2.5507483	1.734883	3.4766548	1.4152566
20220126	2.1835976	2.2426414	2.8831546	1.719067
20220127	2.271618	2.9090185	4.137354	1.7233294
20220128	2.2879848	2.2426414	2.460386	1.8767005

일시	광진구	동대문구	성동구	중랑구
20220101	3.73631	2.71283	2.31818	2.73113
20220102	3.94263	2.43695	2.55403	2.47774
20220103	3.36190	2.95920	2.73224	2.34301
20220104	2.24636	2.52902	2.24619	2.71583
20220105	2.26272	3.22101	2.68843	2.18540
20220106	2.24636	1.85525	2.68843	1.81026
20220107	1.94512	2.27918	2.72871	1.81026
20220108	3.12623	2.01137	1.84096	1.51553
20220109	3.27194	2.95920	1.51318	2.05447
20220110	3.31588	2.01137	2.60947	2.47774
20220111	2.24636	2.13730	3.21112	2.31187
20220112	1.86691	2.01137	2.78736	3.12294
20220113	2.27162	2.03495	2.91807	1.81026
20220114	2.27043	2.24264	3.06688	1.31591
20220115	3.02469	2.95920	1.86650	3.02170
20220116	3.94263	2.41666	1.86650	2.52548
20220117	1.94854	1.91247	2.74635	1.37022
20220118	1.92875	1.91247	3.09752	1.81026
20220119	2.27043	1.89403	3.40034	1.41526
20220120	3.69481	2.24264	2.65629	1.94267
20220121	2.18360	2.24264	3.99407	2.26198
20220122	3.12623	3.46739	3.49155	3.26232
20220123	3.27194	2.71283	3.49155	2.71583
20220124	2.38234	1.93652	2.81508	1.71091

20220125	2.55075	1.73488	3.47665	1.41526
20220126	2.18360	2.24264	2.88315	1.71907
20220127	2.27162	2.90902	4.13735	1.72333
20220128	2.28798	2.24264	2.46039	1.87670
20220129	3.14260	2.95920	3.75518	3.74390
20220130	3.88202	2.71283	3.49155	2.18540
20220131	2.86966	3.50286	3.30505	2.68611
20220201	3.94263	3.85414	2.31818	2.30562
20220202	3.94263	3.22101	2.55403	2.28475
20220203	1.92875	1.85525	2.98569	1.30056
20220204	1.92875	2.09218	2.74263	1.51553
20220205	3.66366	2.03542	2.57979	1.29068
20220206	3.73631	2.32671	1.69995	3.74390
20220207	3.79094	2.11326	2.97474	2.31187
20220208	2.71065	3.49143	2.68843	2.28475
20220209	1.94512	2.11326	2.25960	2.16997
20220210	2.27043	1.85525	3.34876	1.81026
20220211	1.87425	1.71084	3.32855	1.51553
20220212	2.87863	3.46739	2.57979	3.74390
20220213	3.12623	2.95920	2.48526	3.02170
20220214	1.94854	1.91247	2.88772	1.30056
20220215	2.02406	1.94981	2.42589	1.53618
20220216	1.86691	2.03495	3.32806	1.81026
20220217	2.55075	2.24264	3.47665	1.69101

20220218	2.27043	2.24264	3.40034	1.20000
20220219	2.87863	3.46739	3.49155	1.39332
20220220	3.12623	4.49102	3.55535	3.12999
20220221	1.79139	2.18362	2.74635	1.45825
20220222	2.55075	1.73488	3.40034	1.81026
20220223	2.27043	2.24264	4.04575	1.59166
20220224	2.18360	3.04170	4.04575	2.18849
20220225	2.18360	3.04170	3.52828	2.54848
20220226	4.51106	5.07907	2.55784	3.29997
20220227	4.51106	4.13325	3.81898	3.09393
20220228	2.27980	2.78199	3.67801	1.74686
20220301	3.34850	2.62261	4.50338	1.30056
20220302	6.75257	5.70853	6.48159	2.98434
20220303	6.80193	5.56275	5.47222	2.83204
20220304	6.91070	6.79209	5.47137	3.03444
20220305	7.50855	5.15735	5.40916	1.94137
20220306	6.96380	6.79209	5.57313	2.11865
20220307	6.71989	5.56275	5.65917	2.83204
20220308	6.40940	5.11256	4.94320	2.63340
20220309	6.96082	5.56275	5.12122	2.63340
20220310	6.91070	5.68301	5.34808	1.82255
20220311	6.48172	5.56275	5.11155	3.01089
20220312	7.32505	5.56275	4.84548	2.98869
20220313	6.87117	5.56275	5.59729	1.82255
20220314	6.82198	6.79209	5.48611	2.83204

20220315	6.96380	6.79209	5.88869	2.83204
20220316	6.91443	5.56275	5.29477	2.83204
20220317	6.40940	6.79209	5.16380	2.83204
20220318	6.40940	5.56275	5.11155	2.66062
20220319	7.22097	6.79503	8.11125	2.09367
20220320	6.06947	6.64201	8.15338	2.19407
20220321	7.66704	6.18413	5.36518	3.01089
20220322	6.96380	5.95444	5.53621	1.82255
20220323	6.78984	7.68267	7.42456	1.94137
20220324	5.40727	6.56240	6.94009	1.91976
20220325	6.91070	7.33593	5.53621	2.38472
20220326	7.85336	8.55249	9.83452	2.38567
20220327	10.43997	7.76228	9.83452	2.27368
20220328	5.51597	7.30667	5.10971	1.84562
20220329	7.97708	8.47288	7.58302	1.78795
20220330	7.12282	7.17133	7.69167	1.78795
20220331	6.61006	7.22994	5.44193	1.99811
20220401	7.25446	7.33593	7.76315	1.45496
20220402	7.39626	5.71148	5.59236	1.94137
20220403	6.25096	7.68267	5.77814	1.85494
20220404	7.37406	6.49734	5.17926	1.82255
20220405	7.25446	5.72823	5.30935	1.82255
20220406	7.25446	6.43985	7.35927	2.23746
20220407	5.40727	7.68267	5.22576	2.23283
20220408	7.61570	7.68267	5.69699	2.23283

20220409	6.84051	5.56275	5.44406	2.98869
20220410	7.25446	5.56275	5.60681	2.8778
20220411	7.15993	7.68267	5.78585	2.29249
20220412	6.96082	6.91356	5.53621	1.45496
20220413	5.42086	7.30667	5.07702	2.00593
20220414	7.50855	6.79209	5.23379	1.82255
20220415	6.91070	5.68301	5.12122	1.82255
20220416	5.40727	7.07887	5.42654	1.59813
20220417	5.48409	6.49734	7.15320	2.04912
20220418	7.16433	5.68301	5.68906	2.8778
20220419	6.91070	5.07505	5.26839	2.8778
20220420	6.73965	6.90245	7.45454	2.40633
20220421	5.96905	7.33593	5.23597	2.08557
20220422	7.23242	6.24085	5.99643	2.10454
20220423	10.13685	6.68887	7.67960	2.00593
20220424	9.42067	7.22994	7.63092	2.24425
20220425	5.49738	7.22994	7.77319	0.59084
20220426	6.42664	8.97115	7.31168	1.94826
20220427	7.12282	9.20584	7.41198	1.94826
20220428	6.42664	8.85291	6.02838	2.21117
20220429	5.40727	9.20584	7.62557	2.18658
20220430	10.13685	9.37251	7.48645	2.38351
20220501	6.62965	6.79209	7.50270	2.22728
20220502	7.32505	6.43985	4.08909	1.92022
20220503	7.04342	6.91356	5.59128	2.04316

20220504	7.68454	8.47288	5.96098	2.71379
20220505	9.32252	7.04781	7.56984	1.82401
20220506	5.86036	7.43694	7.59585	0.66784
20220507	5.48409	6.18413	6.04092	2.25907
20220508	8.29718	6.99926	5.66780	2.10454
20220509	5.59972	7.68267	7.32736	2.08557
20220510	6.78984	8.26218	6.18227	1.24080
20220511	7.04342	9.37251	4.94168	2.32547
20220512	7.30240	8.26218	4.08909	2.18658
20220513	6.73965	7.68267	5.42025	2.04177
20220514	5.48409	7.68267	5.59729	1.35227
20220515	7.32502	6.79503	8.34996	1.95063
20220516	5.51597	6.72797	7.42481	1.45496
20220517	6.25096	6.92997	5.54864	1.92104
20220518	7.30240	7.17133	6.95593	2.22263
20220519	7.61290	8.79663	5.36518	1.72769
20220520	6.78386	8.26218	5.49800	1.22051
20220521	8.86759	8.51367	7.75274	2.32547
20220522	9.32252	7.30667	7.75274	2.49080
20220523	5.49738	6.13334	7.52082	2.23283
20220524	7.04342	8.97115	4.94168	1.94826
20220525	6.48424	10.32860	7.48794	2.30927
20220526	4.56956	8.78331	7.56984	2.26780
20220527	4.50249	8.78331	6.38599	2.38893
20220528	4.22242	8.25151	6.25152	1.21272

20220529	9.78698	8.85291	7.48645	3.17221
20220530	8.00911	8.47288	5.47964	2.27240
20220531	10.00521	8.78331	6.44007	2.09010
20220601	9.34637	8.55872	7.74182	4.26364
20220602	9.90831	9.40577	7.46480	4.94624
20220603	9.90706	10.10467	7.42537	4.94382
20220604	6.00715	8.97115	5.77729	2.26884
20220605	6.01347	7.66930	5.46329	2.33007
20220606	6.89853	6.43985	5.79271	2.01836
20220607	4.56956	7.38888	7.25585	5.04162
20220608	9.32252	6.71429	7.61852	1.59307
20220609	9.90831	9.24939	6.61111	4.74932
20220610	10.61125	9.62844	7.49368	5.07035
20220611	9.32252	10.10121	5.95889	5.09930
20220612	10.56736	8.83670	7.68664	1.31658
20220613	7.78269	8.78331	7.49460	2.17561
20220614	7.989	5.94898	5.72678	2.26780
20220615	9.32252	9.62844	5.89781	4.87547
20220616	6.26773	9.40552	7.61852	4.69021
20220617	9.32252	10.15933	7.25434	5.10800
20220618	9.90706	10.10121	5.50300	5.12207
20220619	7.96617	9.91905	7.64409	1.79420
20220620	4.51155	7.29532	5.47964	2.45034
20220621	7.96189	7.23532	5.78957	2.45034

20220622	6.54997	7.30667	5.14436	1.72769
20220623	6.84051	7.33593	5.34808	2.08557
20220624	6.78984	7.43694	5.51205	1.45496
20220625	7.96617	6.91029	6.02205	1.85187
20220626	8.07418	9.68876	4.32120	1.79420
20220627	7.30240	8.68505	5.33933	2.32708
20220628	6.91070	6.72797	5.11155	1.92022
20220629	6.48172	5.11256	5.16870	2.83204
20220630	6.48172	5.70853	4.99564	2.83204
20220701	9.34637	9.42095	6.67218	4.94624
20220702	7.24216	7.43694	4.87952	1.18445
20220703	7.44437	7.76228	7.80596	2.11312
20220704	5.40727	6.92997	5.36286	1.45496
20220705	5.51597	3.37365	7.60435	2.32547
20220706	8.39533	7.29532	7.37991	2.50801
20220707	9.32252	9.62844	5.11271	4.83941
20220708	10.61125	9.24939	7.56984	5.23851
20220709	7.20800	8.68505	7.68466	2.26780
20220710	5.64038	7.30667	5.62604	2.27242
20220711	5.40727	9.20584	5.65446	1.94826
20220712	5.51597	6.01636	5.96931	2.17134
20220713	5.51597	9.24939	7.35357	4.89708
20220714	6.26773	9.21099	7.39881	5.10242
20220715	9.32252	9.62844	7.69167	4.92406
20220716	4.56956	8.85291	7.71087	2.52172

20220717	9.94034	9.06314	5.55908	2.88153
20220718	6.69179	9.68876	7.49460	2.82516
20220719	5.40727	8.43474	5.10971	5.12802
20220720	6.47337	9.37251	5.23379	2.23328
20220721	6.47337	8.97115	5.23379	2.39015
20220722	6.47337	9.37251	5.38299	2.65612
20220723	9.32252	8.85291	5.65046	2.11367
20220724	7.76971	8.85291	7.68664	2.38678
20220725	6.91070	10.32860	5.78353	1.25891
20220726	6.40940	6.92997	5.11155	1.92104
20220727	6.58075	5.56275	4.48996	2.66062
20220728	6.58075	6.99926	5.16870	1.82255
20220729	6.91070	7.43694	5.11155	1.89682
20220730	10.56736	9.37251	7.68664	2.28224
20220731	8.55028	8.68505	5.55908	1.87830
20220801	7.40897	8.26218	5.11155	1.92104
20220802	6.78984	7.06557	5.51205	1.24212
20220803	5.40727	8.51367	5.69784	2.00593
20220804	6.50137	9.06778	7.37991	2.31829
20220805	6.26773	6.91029	7.82332	2.41390
20220806	6.55855	7.33593	5.25443	1.85494
20220807	6.80440	6.99926	5.25443	1.96460
20220808	7.30403	7.68267	3.70147	1.96676
20220809	7.27745	8.01540	4.93849	1.82401
20220810	6.91121	8.68505	4.90783	2.26884

20220811	6.31795	8.62046	5.51205	2.79500
20220812	6.48424	8.09561	7.09212	5.09930
20220813	6.58310	7.43694	5.44320	1.27685
20220814	5.56356	7.43694	5.23155	1.78580
20220815	6.94200	3.42351	4.34609	2.65754
20220816	7.77022	8.49350	5.49800	1.61469
20220817	5.40727	8.49350	5.42025	5.09930
20220818	6.78984	6.91029	5.21627	2.88369
20220819	4.51155	8.62046	7.60435	2.50188
20220820	7.12282	8.97115	5.55908	2.57907
20220821	6.40313	8.97115	5.57313	2.09640
20220822	6.78386	8.49350	5.81721	1.78725
20220823	6.74884	6.91029	7.87475	2.30927
20220824	6.48172	6.79209	5.16870	3.01089
20220825	6.48172	6.99926	4.97200	1.82039
20220826	6.91070	7.41699	5.11155	1.45496
20220827	9.32252	8.97115	7.64721	2.21117
20220828	7.20800	7.17133	5.44406	2.38800
20220829	6.47337	7.30667	5.54860	1.78795
20220830	6.91070	6.72797	4.93849	2.23283
20220831	6.58075	7.33593	4.99564	2.40633
20220901	7.15212	8.78331	7.15320	2.51481
20220902	6.49681	10.32860	6.44707	2.26780
20220903	5.63189	8.97115	5.65046	2.32708
20220904	7.12282	8.47288	6.10680	2.18442

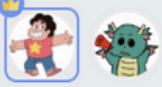
20220905	6.82975	8.78331	7.63056	2.26780
20220906	6.73965	8.21529	7.15430	1.94826
20220907	5.51597	6.91029	7.18804	1.25891
20220908	8.34439	8.09561	7.61852	5.09930
20220909	7.18869	6.49734	5.34808	1.59813
20220910	5.51597	7.68267	5.28683	1.78536
20220911	5.40727	6.99926	5.59729	2.06339
20220912	8.20015	7.30667	6.41827	1.29847
20220913	9.35501	8.96293	7.31563	5.68875
20220914	8.34439	9.21099	6.33411	6.77593
20220915	6.36588	10.15933	7.85054	5.13163
20220916	9.36973	9.40577	7.42537	4.96011
20220917	9.90232	8.55872	6.67478	4.03954
20220918	9.30780	8.49350	6.74258	4.92406
20220919	9.95427	10.10467	7.31563	4.94382
20220920	8.81665	8.89554	7.31563	5.26231
20220921	10.18779	10.15933	6.44007	4.94382
20220922	9.35501	10.10467	7.32265	5.26231
20220923	10.13685	9.21099	7.36612	4.94382
20220924	9.35501	8.55872	7.64409	4.89708
20220925	6.26773	6.99526	7.67960	5.04162
20220926	9.90232	9.24939	7.74079	4.98173
20220927	9.42067	10.10467	6.51362	4.74789
20220928	9.32252	10.10467	7.25434	4.94382

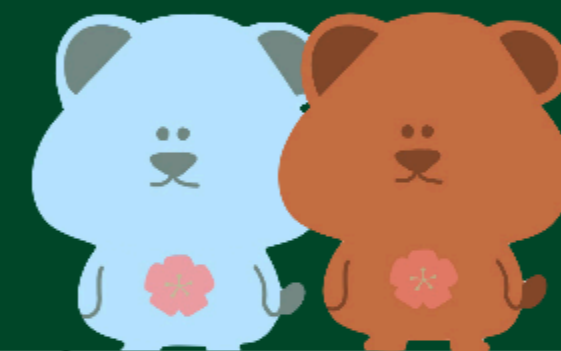
20220929	10.66331	10.10407	7.08389	5.24069
20220930	9.36973	9.40577	7.36612	5.11001
20221001	8.38236	7.30667	7.08586	2.15960
20221002	7.88804	9.05924	7.56113	2.17293
20221003	7.989	7.43694	7.58302	1.78795
20221004	5.42086	8.13157	7.48563	2.11367
20221005	7.12282	10.26654	4.32120	2.26780
20221006	8.38236	9.40577	7.64409	4.92406
20221007	8.34654	9.62844	7.63092	3.98187
20221008	5.52320	8.68505	7.57210	2.31829
20221009	11.06721	8.82898	10.03376	2.26780
20221010	6.42664	9.91905	7.41198	1.68396
20221011	7.22097	9.70131	7.56984	5.14405
20221012	8.29718	9.24939	7.58302	4.80700
20221013	6.75175	9.29925	7.54569	5.07035
20221014	10.56736	8.09561	7.72864	5.04162
20221015	9.07085	10.32860	7.36603	2.45034
20221016	8.38236	6.91029	7.64826	2.58772
20221017	7.22097	9.62844	7.39881	3.98187
20221018	6.78377	9.35067	7.58302	4.92406
20221019	7.96189	10.10121	7.09212	2.48639
20221020	7.33351	8.49350	4.40487	5.04162
20221021	7.97708	8.62046	5.99106	2.50188
20221022	6.50137	8.82898	7.64409	2.52172
20221023	4.58767	8.68505	4.43832	2.30711

20221024	10.18779	8.55872	7.52063	2.53793
20221025	9.30780	9.24939	7.61852	4.93292
20221026	7.68454	8.77020	7.25585	4.92149
20221027	7.96189	8.09561	4.03326	5.04162
20221028	4.50249	8.62046	7.54379	2.50188
20221029	8.29718	8.68505	8.04392	2.38893
20221030	7.20800	8.97115	7.44099	2.20901
20221031	4.61677	8.43474	5.85735	4.97916
20221101	6.78984	9.06778	4.32120	2.28224
20221102	5.76203	9.12821	7.66542	2.32708
20221103	6.50137	8.78331	5.95889	2.28224
20221104	7.76869	8.49350	7.75274	2.50188
20221105	5.40727	8.47288	8.04392	2.18658
20221106	6.89593	8.97115	7.66143	2.49080
20221107	6.78984	8.93349	6.38599	2.60624
20221108	6.77171	6.79390	7.70425	1.90408
20221109	5.99265	8.09561	9.26581	1.78725
20221110	7.04342	9.62844	6.30239	5.20245
20221111	6.48424	8.55872	7.63092	2.45034
20221112	7.68454	8.68505	7.64826	1.87830
20221113	5.49245	8.74987	7.08586	2.26564
20221114	6.42664	8.09561	7.41198	2.50188
20221115	5.48409	8.09561	4.40487	1.79420
20221116	6.02325	8.78331	5.59729	2.51481
20221117	7.04342	8.78331	5.53621	2.28224

20221118	5.99512	8.82898	7.78936	2.57907
20221119	6.27151	9.37251	7.46022	2.51481
20221120	6.05271	8.47288	7.66644	2.71204
20221121	7.22097	10.10121	6.92109	2.88369
20221122	6.31795	10.32860	7.76309	2.26780
20221123	5.86036	8.68505	5.49800	2.36314
20221124	5.89643	9.12821	5.15612	2.09855
20221125	6.78984	8.68505	7.68664	2.57907
20221126	6.82482	7.63832	7.68466	2.18442
20221127	5.56356	7.30667	8.34996	1.94558
20221128	6.12216	8.69860	4.08909	2.35623
20221129	6.73965	9.91905	8.05018	2.03226
20221130	6.34139	9.91905	9.65747	2.26780

최종 성능 결과

#	팀	팀 멤버	점수	제출수	등록일
103	*		3.44385	1	몇 초 전
1	3S		1.24113	57	3년 전
2	하돌이		1.24113	29	2년 전
3	b612		1.25606	64	3년 전
4	맘맘마		1.25868	15	3년 전



감사합니다

장인서 백재은 유연 정윤지